Solution: Problem Challenge 1: Tree Diameter

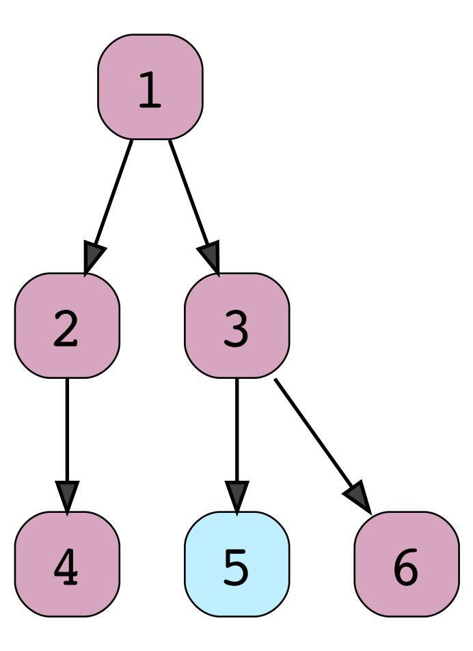
Problem Statement

Given a binary tree, find the length of its diameter. The diameter of a tree is the number of nodes on the **longest path between any two leaf nodes.** The diameter of a tree may or may not pass through the root.

Note: You can always assume that there are at least two leaf nodes in the given tree.

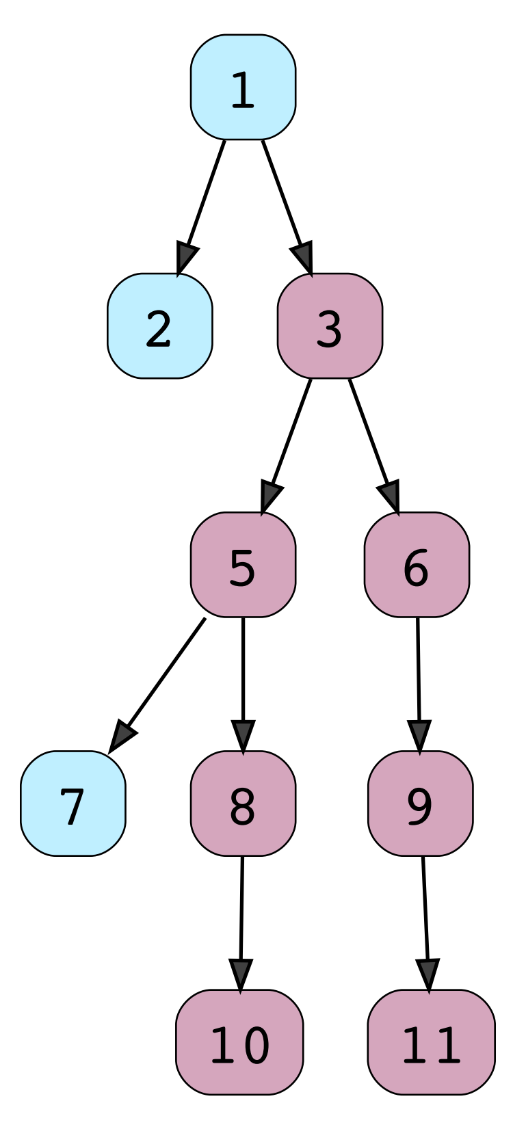
Example 1:

Output: 5
Explanation: The diameter of the tree is: [4, 2, 1, 3, 6]



Example 2:

Output: 7
Explanation: The diameter of the tree is: [10, 8, 5, 3, 6, 9, 11]



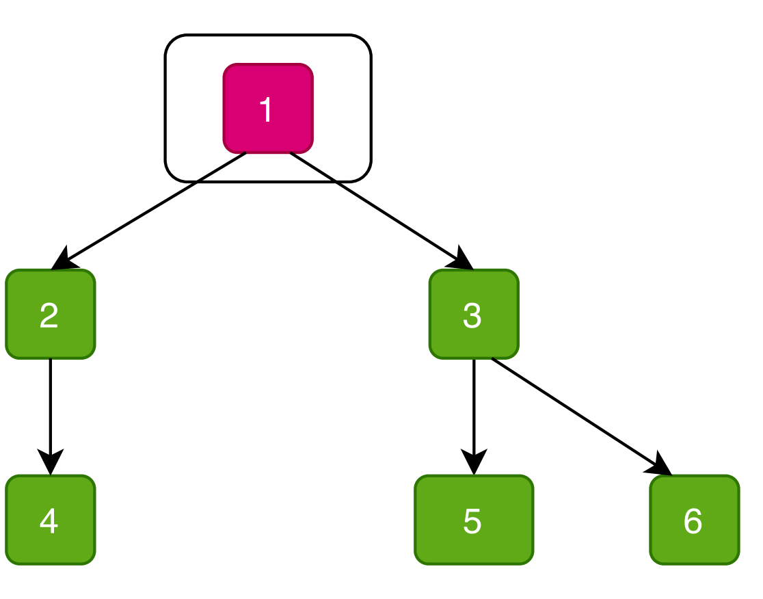
Solution

This problem follows the **Binary Tree Path Sum** pattern. We can follow the same **DFS** approach. There will be a few differences:

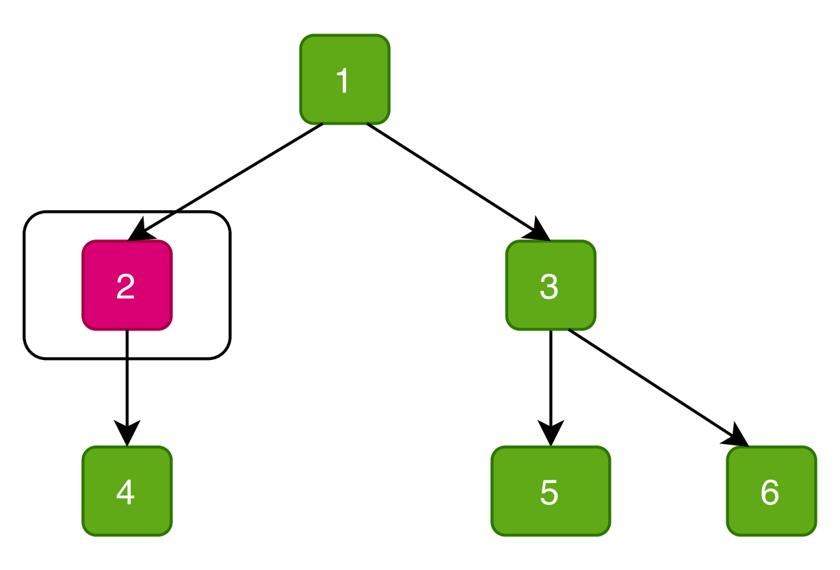
1. At every step, we need to find the height of both children of the current node. For this, we will make two recursive calls similar to **DFS**.
2. The height of the current node will be equal to the maximum of the heights of its left or right children, plus '1' for the current node.
3. The tree diameter at the current node will be equal to the height of the left child plus the height of the right child plus '1' for the current node: `diameter = leftTreeHeight + rightTreeHeight + 1`. To find the overall tree diameter, we will use a class level variable. This variable will store the maximum diameter of all the nodes visited so far, hence, eventually, it will have the final tree diameter.

Here is the visual representation of the algorithm:

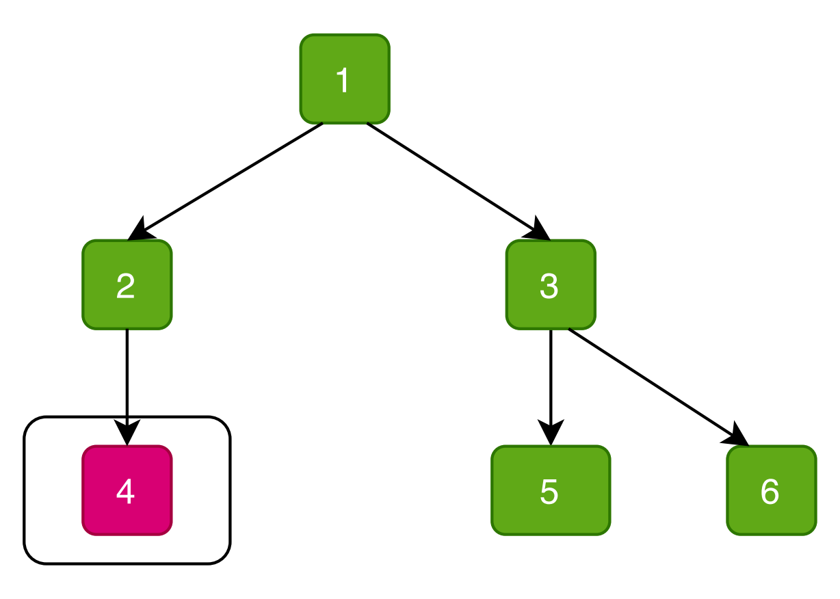
LeftTreeHeight: ∞
RightTreeHeight: ∞
TreeDiameter: 0



LeftTreeHeight: ∞
RightTreeHeight: ∞
TreeDiameter: 0



LeftTreeHeight: 0
RightTreeHeight: 0
TreeDiameter: 0



Since 4 is the leaf node and NULL will be passed to CalculateHeight function as left and right of 4 so now exit condition will be called and 0 will be returned

As a result, LeftTreeHeight and RightTreeHeight will have 0.
Whenever a value is set to LeftTreeHeight and RightTreeHeight, we will check:

if both are not 0 then:

diameter = LeftTreeHeight + RightTreeHeight + 1(to include currentNode too)

TreeDiameter=GetMax(TreeDiameter, diameter)

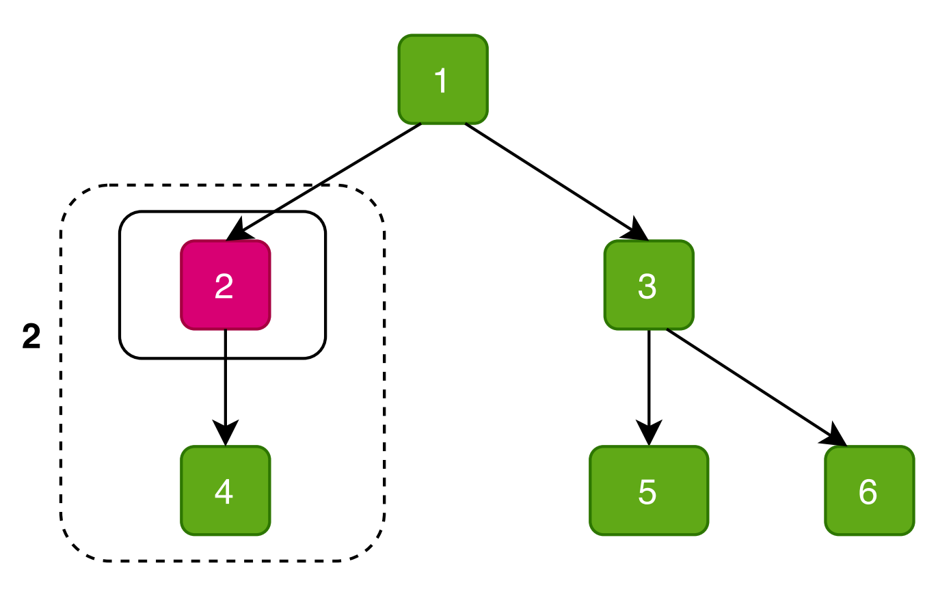
return GetMax(LeftTreeHeight,RightTreeHeight) + 1

According to above algo, 1 will be returned because left and right tree height have 0 at node 4

Now we will backtrack to 2 and LeftTreeHeight=1 and RightTreeHeight=0 as 2 has no right node

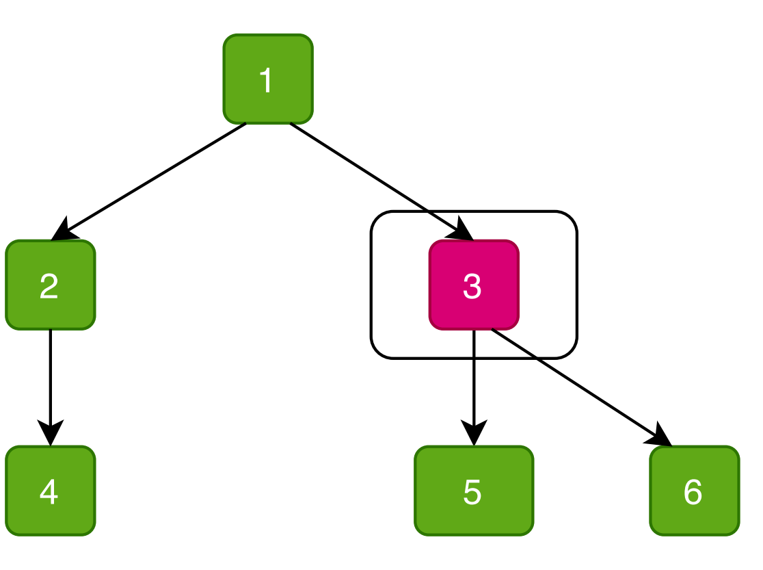
LeftTreeHeight: 2
RightTreeHeight: 0
TreeDiameter: 0

At Node 2, the LeftTreeHeight becomes 2



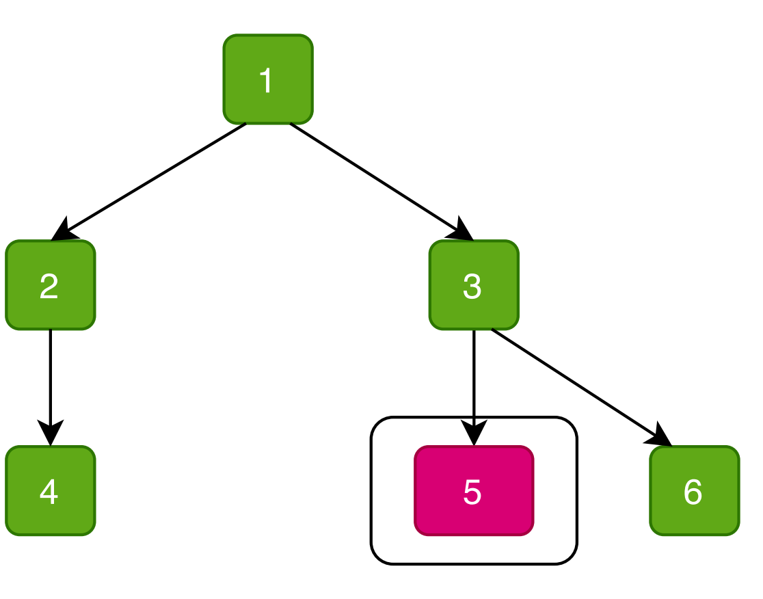
Now, we will backtrack and will move to 3 at the right

LeftTreeHeight: ∞
RightTreeHeight: ∞
TreeDiameter: 0

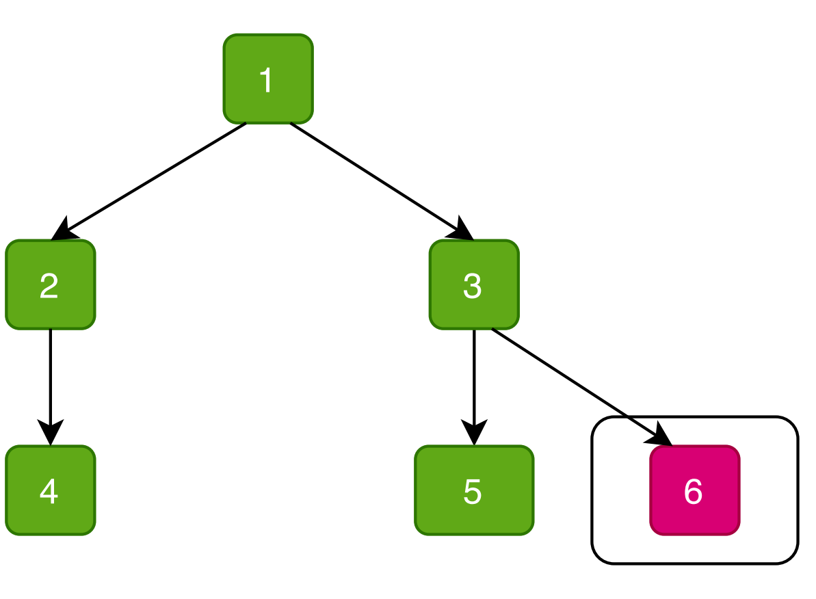


We will follow the same approach at node 5 as we have followed for 4, so:

LeftTreeHeight: 1
RightTreeHeight: ∞
TreeDiameter: 0



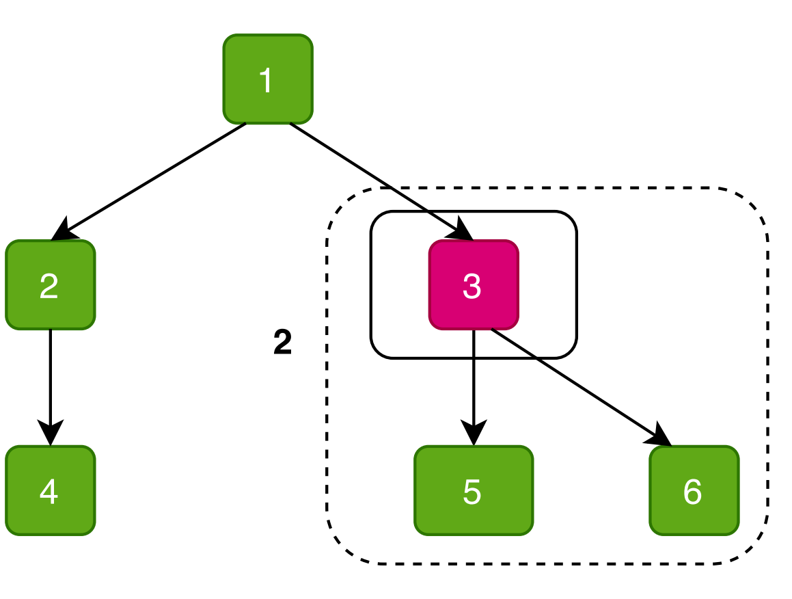
LeftTreeHeight: 0
RightTreeHeight: 1
TreeDiameter: 0



Now, we will backtrack to 3 and at 3, we have

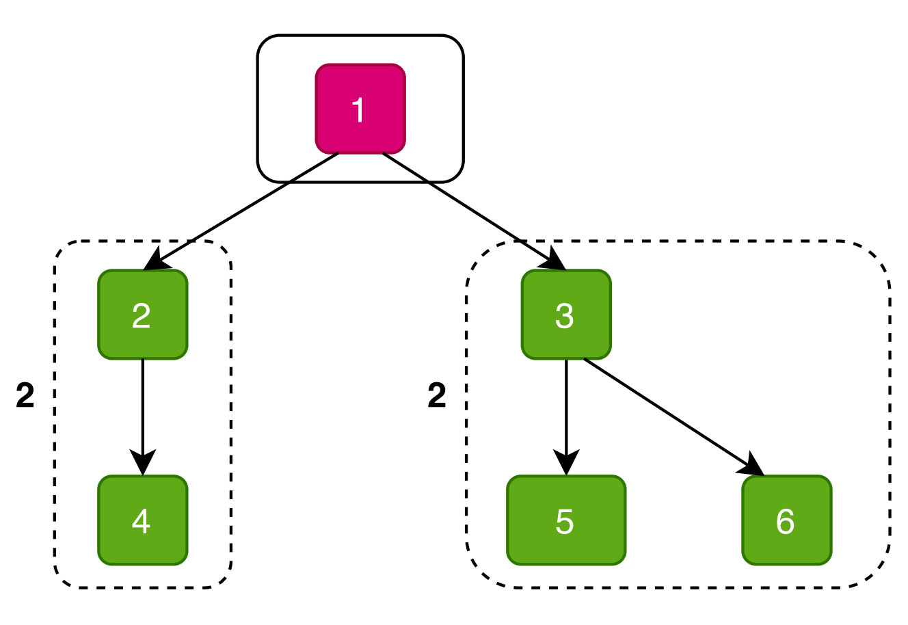
LeftTreeHeight: 1
RightTreeHeight: 1
TreeDiameter: 1+1+1(for the current node)

Hence, max height will be returned from node 3 which is 2



Now, we will backtrack to 1 and at 1, we have
LeftTreeHeight: 2
RightTreeHeight: 2
TreeDiameter: 2+2+1(for the current node)

Hence, max height will be returned from node 1 which is 3 and at last we willexit the function and TreeDiameter=5

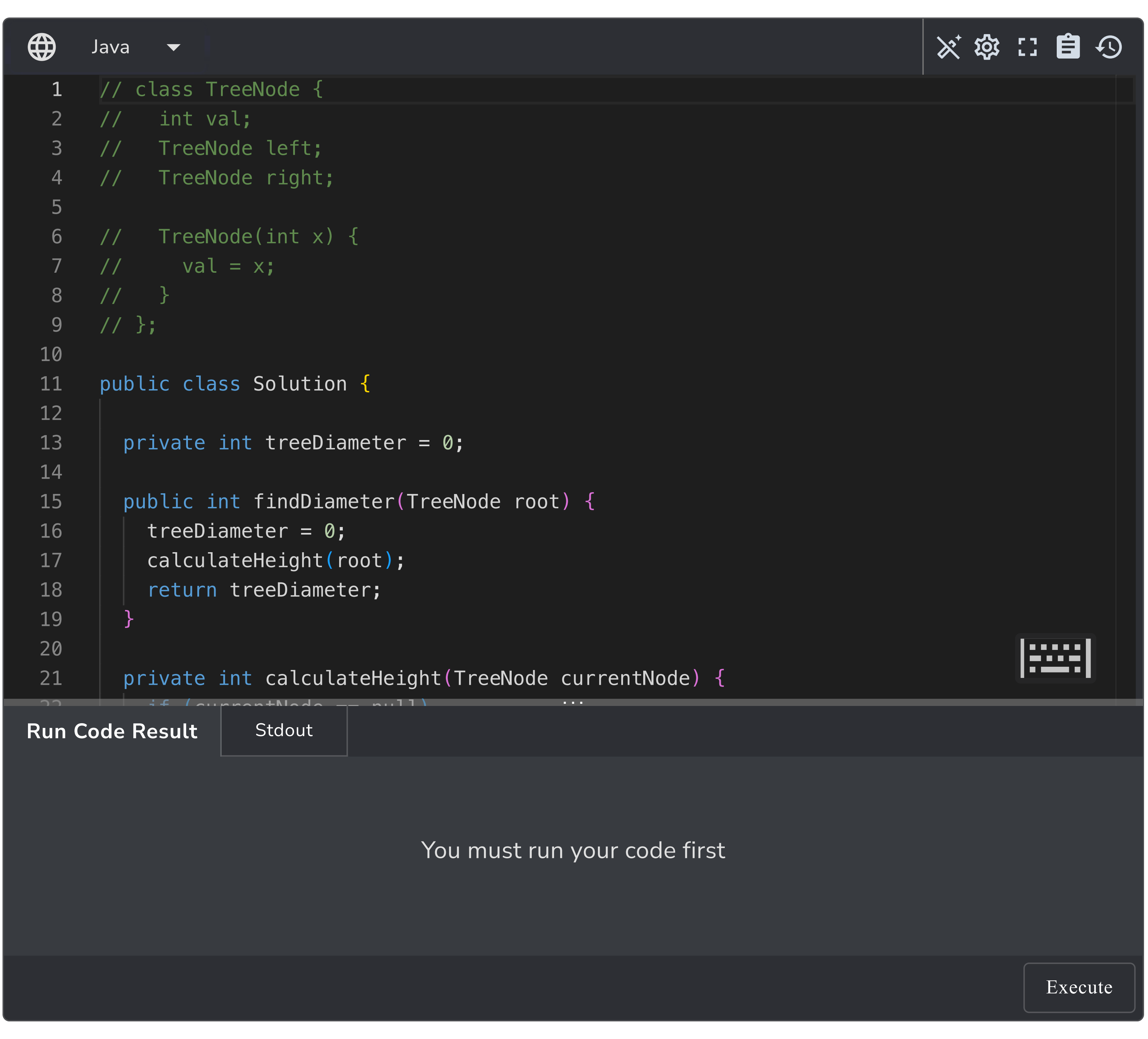


Tree Diameter: 5

Tree Diameter

Code

Here is the code for this algorithm:



```java
// class TreeNode {
//   int val;
//   TreeNode left;
//   TreeNode right;

//   TreeNode(int x) {
//     val = x;
//   }
// };

public class Solution {

  private int treeDiameter = 0;

  public int findDiameter(TreeNode root) {
    treeDiameter = 0;
    calculateHeight(root);
    return treeDiameter;
  }

  private int calculateHeight(TreeNode currentNode) {
```

You must run your code first

Time Complexity

The time complexity of the above algorithm is O(N), where 'N' is the total number of nodes in the tree. This is due to the fact that we traverse each node once.

Space Complexity

The space complexity of the above algorithm will be O(N) in the worst case. This space will be used to store the recursion stack. The worst case will happen when the given tree is a linked list (i.e., every node has only one child).

← Previous
Problem Challenge 1: Tree Diameter (Medium)

Next →
Problem Challenge 2: Path With Maximum Sum (Hard)

☐ Mark as Completed